sir_llm_hackathon_final

March 8, 2026

1 AI-Guided SIR Modeling Hackathon (2 Hours)

This notebook is the **workshop skeleton** for the **AI-Guided SIR Modeling Hackathon**.

- Dataset: **Niamey, Niger measles outbreaks** (biweekly case counts; communities A/B/C)
- Tooling: **Google Colab + Python + ChatGPT**
- Time unit: **biweeks**
- Key modeling shortcut: infectious period **2 weeks** **1 per biweek**

1.1 How to use this notebook

Each section contains: 1) A **ChatGPT prompt** (Markdown) you can copy into ChatGPT
2) A **starter code cell** (often with TODOs) to run in Colab

Rule of thumb: Always request **plots + sanity checks** when you ask ChatGPT for help.

1.2 0. Setup

1.2.1 ChatGPT Prompt (copy/paste)

```
Persona:
You are an expert epidemiological modeler and Python educator.

Task:
- Write a Colab-ready setup cell: imports, basic plotting config, and a helper for reproducibil
- If any package is missing show me how to run pip install

Constraints:
- Use numpy, scipy, pandas, matplotlib only

Verification:
- Print library versions and a simple "ready" message.
```

[1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import integrate, optimize, stats
import sys
```

```python
# Basic plotting configuration
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['axes.grid'] = True
plt.style.use('ggplot')

def set_reproducibility(seed=42):
    """Helper to set seeds for reproducibility."""
    np.random.seed(seed)
    print(f"Seed set to {seed} for reproducibility.")

# Verification
print(f"Python version: {sys.version}")
print(f"NumPy version:  {np.__version__}")
print(f"Pandas version: {pd.__version__}")
print(f"Matplotlib:    {plt.matplotlib.__version__}")

set_reproducibility()
print("\n--- Setup ready ---")
```

```
Python version: 3.12.12 (main, Oct 10 2025, 08:52:57) [GCC 11.4.0]
NumPy version:  2.0.2
Pandas version: 2.2.2
Matplotlib:    3.10.0
Seed set to 42 for reproducibility.

--- Setup ready ---
```

1.2.2 Chat GPT Prompt to generate code to load data

```
Persona:
You are an expert epidemiological modeler and Python educator.

Task:
- Write a Colab-ready python code to:
  - Create `data` folder if it is not exists
  - Download the csv file `https://github.com/merlinvn/ai-guided-modeling-hackathon/raw/refs/he
```

[2]:
```python
import os
import requests

# Create data folder if it does not exist
if not os.path.exists('data'):
    os.makedirs('data')
    print("Created 'data' directory.")

# Download the csv file
url = "https://github.com/merlinvn/ai-guided-modeling-hackathon/raw/refs/heads/
↪main/data/niamey.csv"
```

```
data_path = "data/niamey.csv"

response = requests.get(url)
with open(data_path, 'wb') as f:
    f.write(response.content)

print(f"Downloaded dataset to {data_path}")
```

```
Downloaded dataset to data/niamey.csv
```

1.3 1. Explore the Niamey Measles Data:

1.3.1 Load the dataset and compute basic statistics

ChatGPT Prompt (copy/paste)

```
Persona:
You are an expert epidemiological modeler and Python educator.

Context:
- Dataset contains biweekly measles case counts from Niamey, Niger
- File path: data/niamey.csv

Task:
Write Python code to:
1. Load data/niamey.csv store it to `measles_df`
2. Display the first few rows
3. Print column names and basic information about the dataset
4. Show basic statistics about dataset

Constraints
- Use pandas only
- Keep code simple and readable
```

[3]:
```
import pandas as pd

# 1. Load data/niamey.csv store it to measles_df
measles_df = pd.read_csv('data/niamey.csv')

# 2. Display the first few rows
print("First 5 rows of the dataset:")
display(measles_df.head())

# 3. Print column names and basic information
print("\nDataset Information:")
measles_df.info()

# 4. Show basic statistics about dataset
print("\nBasic Statistics:")
```

```python
display(measles_df.describe())

# Check for missing values
print("\nMissing values per column:")
print(measles_df.isnull().sum())
```

```
First 5 rows of the dataset:

   biweek community  measles
0       1         A     22.0
1       2         A     27.0
2       3         A     64.0
3       4         A     84.0
4       5         A    116.0


Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 48 entries, 0 to 47
Data columns (total 3 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   biweek     48 non-null     int64
 1   community  48 non-null     object
 2   measles    47 non-null     float64
dtypes: float64(1), int64(1), object(1)
memory usage: 1.3+ KB

Basic Statistics:

          biweek      measles
count  48.000000    47.000000
mean    8.500000   232.531915
std     4.658554   316.792200
min     1.000000     0.000000
25%     4.750000    13.500000
50%     8.500000    81.000000
75%    12.250000   251.500000
max    16.000000  1041.000000


Missing values per column:
biweek       0
community    0
measles      1
dtype: int64
```

1.3.2 Exercise

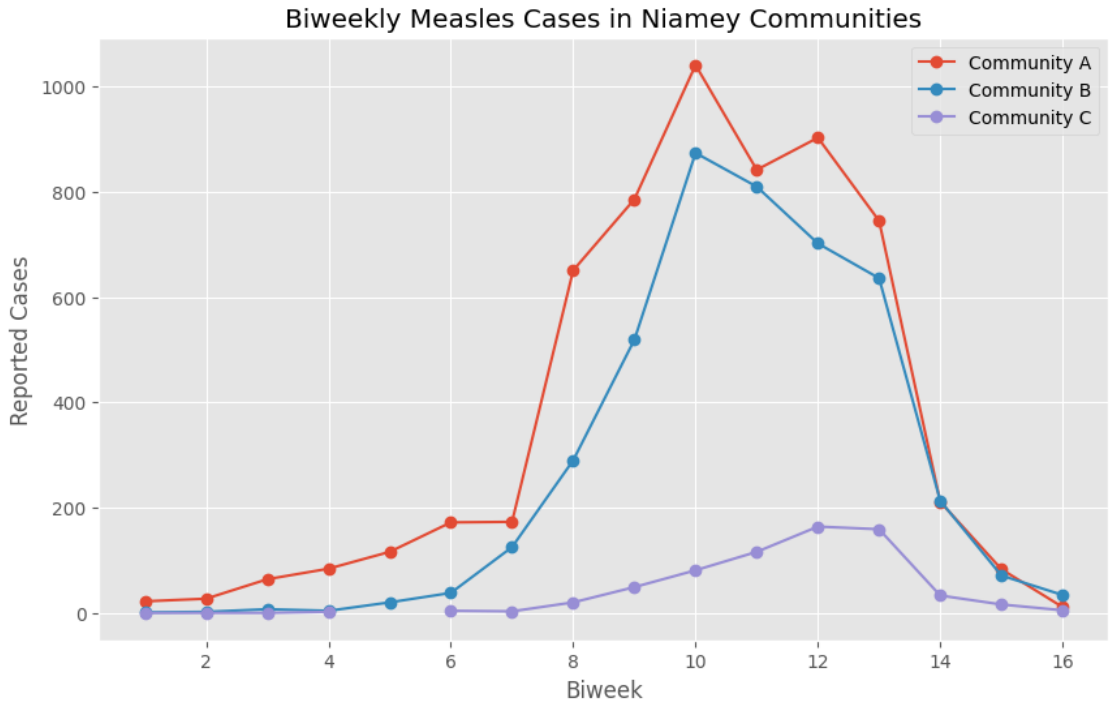
// TODO: Participants write a prompt to explain the results in plain language

1.3.3 Basic Plot

ChatGPT Prompt (copy/paste)

```
Persona:
You are an expert epidemiological modeler and Python educator.

Context:
- Dataset is already loaded into a DataFrame called `measles_df`
- Columns: biweek, community, measles

Task:
- Plot measles cases over time for each community
- Use one time-series plot
- Label axes and include a legend
- Briefly explain what biweekly case counts mean for epidemic modeling
```

[4]:
```python
import matplotlib.pyplot as plt

# Plotting measles cases over time for each community
fig, ax = plt.subplots()

for label, df in measles_df.groupby('community'):
    df.plot(x='biweek', y='measles', ax=ax, label=f'Community {label}',␣
 ↪marker='o')

ax.set_title('Biweekly Measles Cases in Niamey Communities')
ax.set_xlabel('Biweek')
ax.set_ylabel('Reported Cases')
ax.legend()

plt.show()

# Brief explanation
print("Note: Biweekly case counts represent the incidence (new cases) over a␣
 ↪2-week window.")
print("In SIR modeling, if the infectious period is roughly 2 weeks, then the␣
 ↪removal rate (gamma) is ~1 per biweek.")
```




```
Note: Biweekly case counts represent the incidence (new cases) over a 2-week
window.
In SIR modeling, if the infectious period is roughly 2 weeks, then the removal
rate (gamma) is ~1 per biweek.
```

1.4 2. Feature-Based Estimation: Early Growth (Quick R)

We estimate early exponential growth using a semi-log regression: - Choose an early window where cases grow roughly exponentially. - Fit `log(cases)` vs time.

1.4.1 ChatGPT Prompt (copy/paste)

```
## Persona

You are an **expert in infectious disease modeling**, with experience in early-outbreak analysi

## Context

* **Disease:** Measles
* **Community:** A
* **Data:** Biweekly reported case counts
* **Epidemic phase:** Early outbreak (first ~8-10 biweeks), where case counts exhibit **exponen
* **Dataset:** Already loaded as `measles_df`

### Dataset structure
```

```
Dataset Information:
`
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 48 entries, 0 to 47
Data columns (total 3 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   biweek     48 non-null     int64
 1   community  48 non-null     object
 2   measles    47 non-null     float64
dtypes: float64(1), int64(1), object(1)
`

## Task

Estimate the **early exponential growth rate** of the outbreak by:

1. Selecting an appropriate **early biweek window** (~ 8-10 biweeks)
2. Fitting a **log-linear model**: log(measles cases) vs. time
3. Producing a **semi-log plot** showing observed data and fitted exponential growth
4. Converting the estimated growth rate to an **approximate basic reproduction number (R )**, a

   * Infectious removal rate: **gamma = 1 per biweek**

## Constraints

* Use **NumPy** and **SciPy** only (Matplotlib for plotting is acceptable)
* Clearly **state and explain modeling assumptions**
* Print key **statistical results** (e.g., slope, confidence intervals if available, R0)
* **Output code only** (do not execute)

## Verification / Expected Output

* A **semi-log plot** of measles cases with the fitted exponential curve
* Printed values for:

  * Estimated **growth rate (slope)**
  * Corresponding **R0 estimate**
```

[5]:
```python
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

# ----------------------------
# Assumptions
# ----------------------------
```

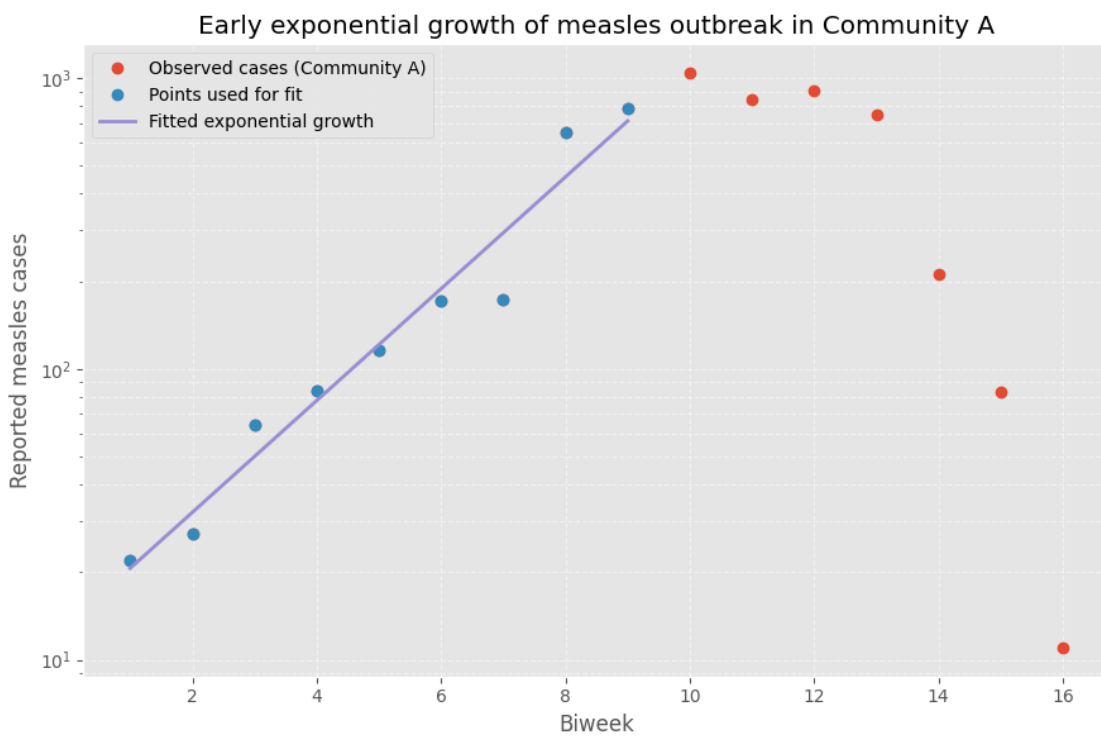
```
# 1. We analyze only Community A.
# 2. Early outbreak dynamics are approximately exponential over the first 8-10␣
↪biweeks.
# 3. Reported case counts in the selected window are positive and suitable for␣
↪log transformation.
# 4. Time is measured in biweeks.
# 5. Under a simple SIR-style approximation with infectious removal rate gamma␣
↪= 1 per biweek,
#    the early exponential growth rate r implies:
#         R0   1 + r / gamma
#    so here:
#         R0   1 + r
#
# 6. The early window is chosen automatically as the first 8␣
↪positive-observation biweeks
#    for Community A. You can adjust `n_early_points` if needed after visual␣
↪inspection.

gamma = 1.0  # per biweek
n_early_points = 9

# ----------------------------
# Prepare data
# ----------------------------
df_a = (
    measles_df.loc[measles_df["community"] == "A", ["biweek", "measles"]]
    .dropna()
    .sort_values("biweek")
    .copy()
)

# Keep only positive counts for log-linear fitting
df_pos = df_a[df_a["measles"] > 0].copy()

if len(df_pos) < n_early_points:
    raise ValueError(
        f"Not enough positive observations in Community A for fitting. "
        f"Found {len(df_pos)}, need at least {n_early_points}."
    )

# Select early outbreak window
early = df_pos.iloc[:n_early_points].copy()

t = early["biweek"].to_numpy(dtype=float)
y = early["measles"].to_numpy(dtype=float)
logy = np.log(y)
```

```python
# ----------------------------
# Fit log-linear model
# log(y) = a + r * t
# ----------------------------
fit = stats.linregress(t, logy)

intercept = fit.intercept
slope = fit.slope                    # early exponential growth rate r (per biweek)
slope_se = fit.stderr
intercept_se = fit.intercept_stderr
r_value = fit.rvalue
p_value = fit.pvalue

# 95% CI for slope
alpha = 0.05
dfree = len(t) - 2
tcrit = stats.t.ppf(1 - alpha / 2, df=dfree)

slope_ci_low = slope - tcrit * slope_se
slope_ci_high = slope + tcrit * slope_se

# Approximate R0
R0_est = 1.0 + slope / gamma
R0_ci_low = 1.0 + slope_ci_low / gamma
R0_ci_high = 1.0 + slope_ci_high / gamma

# ----------------------------
# Fitted values for plotting
# ----------------------------
t_fit = np.linspace(t.min(), t.max(), 200)
logy_fit = intercept + slope * t_fit
y_fit = np.exp(logy_fit)

# ----------------------------
# Print results
# ----------------------------
print("Early exponential growth analysis for measles outbreak in Community A")
print("-" * 65)
print(f"Selected early window (biweeks): {early['biweek'].tolist()}")
print(f"Number of points used: {len(early)}")
print()
print("Log-linear model:")
print("  log(cases) = intercept + slope * biweek")
print(f"  intercept = {intercept:.4f} (SE = {intercept_se:.4f})")
print(f"  slope     = {slope:.4f} per biweek (SE = {slope_se:.4f})")
print(f"  95% CI for slope: [{slope_ci_low:.4f}, {slope_ci_high:.4f}]")
```

```
print(f"  R^2 = {r_value**2:.4f}")
print(f"  p-value for slope = {p_value:.4g}")
print()
print("Approximate basic reproduction number:")
print(f"  gamma = {gamma:.2f} per biweek")
print(f"  R0   1 + slope/gamma = {R0_est:.4f}")
print(f"  95% CI for R0: [{R0_ci_low:.4f}, {R0_ci_high:.4f}]")

# ----------------------------
# Semi-log plot
# ----------------------------
plt.figure(figsize=(9, 6))

# All observed Community A data
plt.semilogy(
    df_pos["biweek"],
    df_pos["measles"],
    "o",
    label="Observed cases (Community A)"
)

# Highlight early fitting window
plt.semilogy(
    early["biweek"],
    early["measles"],
    "o",
    label="Points used for fit"
)

# Fitted exponential curve
plt.semilogy(
    t_fit,
    y_fit,
    "-",
    linewidth=2,
    label="Fitted exponential growth"
)

plt.xlabel("Biweek")
plt.ylabel("Reported measles cases")
plt.title("Early exponential growth of measles outbreak in Community A")
plt.legend()
plt.grid(True, which="both", linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()
```

```
Early exponential growth analysis for measles outbreak in Community A
-------------------------------------------------------------------
```

```
Selected early window (biweeks): [1, 2, 3, 4, 5, 6, 7, 8, 9]
Number of points used: 9

Log-linear model:
  log(cases) = intercept + slope * biweek
  intercept = 2.5843 (SE = 0.1992)
  slope     = 0.4426 per biweek (SE = 0.0354)
  95% CI for slope: [0.3589, 0.5263]
  R^2 = 0.9572
  p-value for slope = 4.821e-06

Approximate basic reproduction number:
  gamma = 1.00 per biweek
  R0   1 + slope/gamma = 1.4426
  95% CI for R0: [1.3589, 1.5263]
```



Goal Assess how sensitive the estimated basic reproduction number (**R**) is to the choice of early-outbreak window length.

Method (brief) For a sequence of increasing early-outbreak windows (measured in biweeks):

- Fit a log-linear model to early reported measles cases to estimate the exponential growth rate.
- Convert the growth rate to **R** assuming a fixed infectious removal rate (= 1 per biweek).
- Compute a 95% confidence interval for **R** from the regression uncertainty.
- Visualize **R** and its uncertainty as a function of the window length.

Interpretation Stable **R** estimates across window lengths suggest a robust early-growth signal, while strong variation indicates sensitivity to window choice or departures from exponential growth.

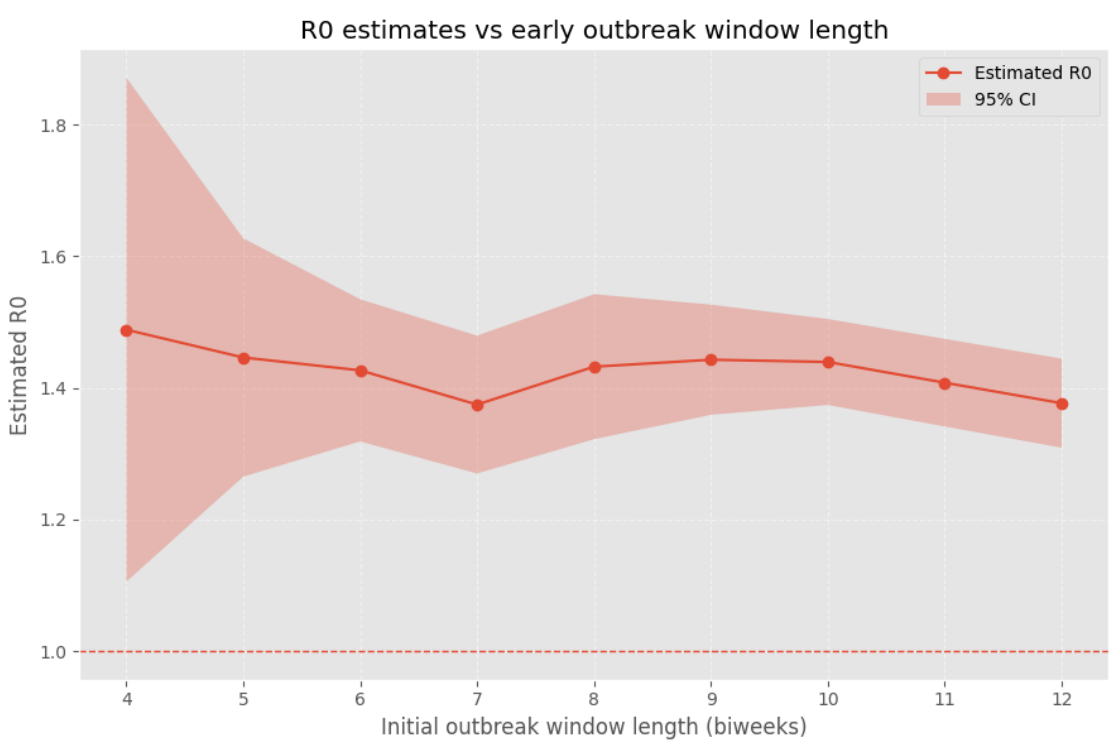
1.4.2 Continue with the this prompt

```
For a range of early-outbreak window lengths, plot the estimated R0 (derived from exponential g
```

[6]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

# ----------------------------
# Assumptions
# ----------------------------
# 1. Early outbreak follows exponential growth.
# 2. log(cases) = a + r*t
# 3. R0   1 + r/gamma
# 4. gamma = 1 per biweek
# 5. We vary the early window length and observe how R0 estimates change.

gamma = 1.0

min_window = 4
max_window = 12

# ----------------------------
# Prepare data
# ----------------------------
df_a = (
    measles_df.loc[measles_df["community"] == "A", ["biweek", "measles"]]
    .dropna()
    .sort_values("biweek")
)

df_a = df_a[df_a["measles"] > 0].copy()

t_all = df_a["biweek"].to_numpy()
y_all = df_a["measles"].to_numpy()

# ----------------------------
# Storage for results
# ----------------------------
window_lengths = []
R0_estimates = []
R0_lower = []
R0_upper = []
```

```python
# ----------------------------
# Loop through early windows
# ----------------------------
for n in range(min_window, max_window + 1):

    if n > len(t_all):
        break

    t = t_all[:n]
    y = y_all[:n]
    logy = np.log(y)

    fit = stats.linregress(t, logy)

    slope = fit.slope
    slope_se = fit.stderr

    dfree = n - 2
    tcrit = stats.t.ppf(0.975, df=dfree)

    slope_low = slope - tcrit * slope_se
    slope_high = slope + tcrit * slope_se

    R0 = 1 + slope / gamma
    R0_low = 1 + slope_low / gamma
    R0_high = 1 + slope_high / gamma

    window_lengths.append(n)
    R0_estimates.append(R0)
    R0_lower.append(R0_low)
    R0_upper.append(R0_high)

# Convert to arrays
window_lengths = np.array(window_lengths)
R0_estimates = np.array(R0_estimates)
R0_lower = np.array(R0_lower)
R0_upper = np.array(R0_upper)

# ----------------------------
# Plot
# ----------------------------
plt.figure(figsize=(9,6))

plt.plot(window_lengths, R0_estimates, marker='o', label="Estimated R0")

plt.fill_between(
```

```
    window_lengths,
    R0_lower,
    R0_upper,
    alpha=0.3,
    label="95% CI"
)

plt.axhline(1, linestyle="--", linewidth=1)

plt.xlabel("Initial outbreak window length (biweeks)")
plt.ylabel("Estimated R0")
plt.title("R0 estimates vs early outbreak window length")
plt.grid(True, linestyle="--", alpha=0.5)
plt.legend()

plt.tight_layout()
plt.show()

# ------------------------------
# Print results table
# ------------------------------
print("Window\tR0\tLower95\tUpper95")
for w, r, lo, hi in zip(window_lengths, R0_estimates, R0_lower, R0_upper):
    print(f"{w}\t{r:.3f}\t{lo:.3f}\t{hi:.3f}")
```



```
Window  R0      Lower95 Upper95
4       1.488   1.105   1.871
5       1.446   1.265   1.627
6       1.426   1.319   1.534
7       1.374   1.270   1.479
8       1.432   1.322   1.542
9       1.443   1.359   1.526
10      1.439   1.374   1.504
11      1.408   1.341   1.474
12      1.377   1.309   1.444
```

Using the above results, ask yourself and ChatGPT the follow up questions:

- Which early window length gives the most stable estimate of R0?
- Why might very short windows (e.g., 4 biweeks) produce less reliable estimates?
- What happens if you extend the window too far into the epidemic?
- Ask ChatGPT to modify the code to compute the variance of the R0 estimate across window lengths and identify the most stable region.

Then later (homeworks):

- Does the growth rate differ between communities?
- Which community had the fastest outbreak growth?
- Are the R0 estimates consistent across communities?
- Ask ChatGPT to repeat the full analysis for communities A, B, and C and plot the estimated R0 curves on the same figure.

1.5 3. Implement the SIR Model (Biweekly Units)

We use a **closed SIR** model for a single outbreak wave: - S(t): susceptible - I(t): infectious - R(t): recovered

Time unit = **biweeks**.

1.5.1 ChatGPT Prompt (copy/paste)

```
Persona:
You are an expert epidemiological modeler and Python educator.

Context:
- Closed SIR model (no birth or death)
- Time unit: biweeks
- Infectious period is around 2 weeks (gamma is 1 per biweek)

Task:
Implement an SIR model using SciPy ODE solving:
```

```
- Define the SIR equations, the function name should be `sir_rhs`
- Solve over a biweekly time grid, put it in the function `simulate_sir`
- The initial parameters should include:
    - N: population size
    - I0: initial infections
    - immune_fration: fraction of population are immune
    - beta: transmission rate
    - gamma: recovery rate
- Plot S, I, R over time

Constraints:
- Use scipy.integrate.solve_ivp
- Keep code readable and commented

Verification:
- Plot S, I, R
- Verify S + I + R ~ N
```

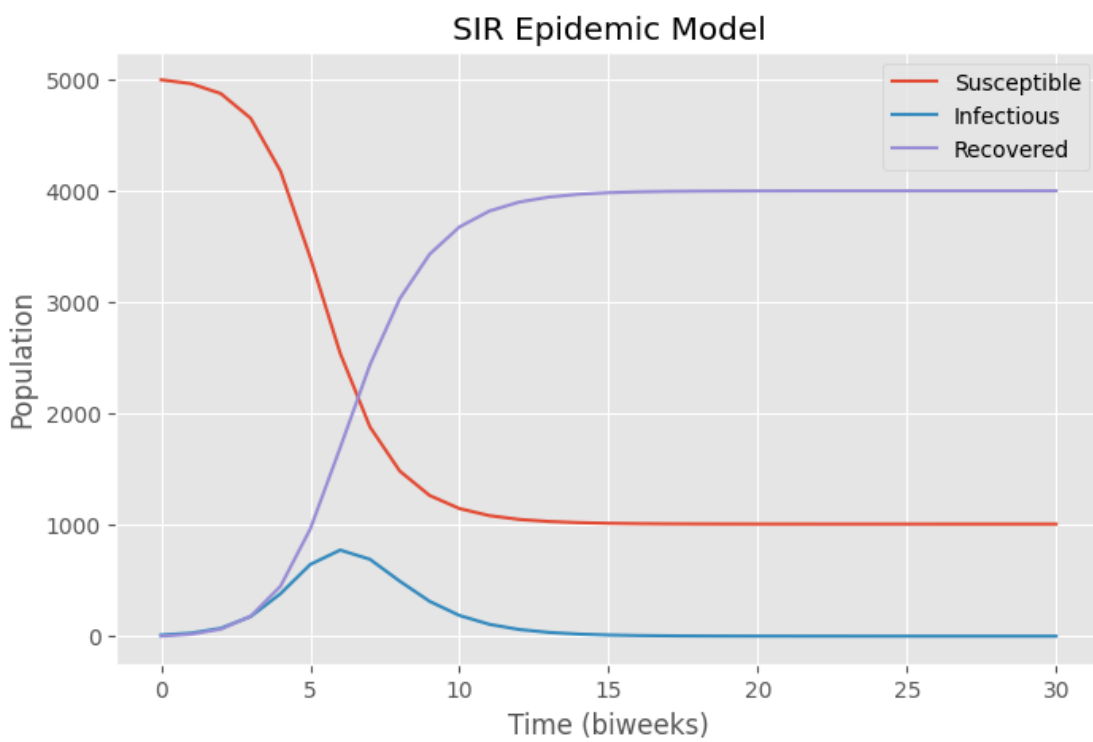
[7]:
```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp


def sir_rhs(t, y, beta, gamma, N):
    """
    Right-hand side of the SIR differential equations.

    Parameters
    ----------
    t : float
        Time (in biweeks)
    y : array-like
        Current state [S, I, R]
    beta : float
        Transmission rate
    gamma : float
        Recovery rate (per biweek)
    N : int
        Total population

    Returns
    -------
    dydt : list
        Time derivatives [dS/dt, dI/dt, dR/dt]
    """

    S, I, R = y
```

```python
    dSdt = -beta * S * I / N
    dIdt = beta * S * I / N - gamma * I
    dRdt = gamma * I

    return [dSdt, dIdt, dRdt]


def simulate_sir(N, I0, immune_fraction, beta, gamma,
                 t_max=30, dt=1):
    """
    Simulate an SIR model over a biweekly time grid.

    Parameters
    ----------
    N : int
        Total population
    I0 : int
        Initial number of infections
    immune_fraction : float
        Fraction of population initially immune
    beta : float
        Transmission rate
    gamma : float
        Recovery rate
    t_max : int
        Simulation duration (biweeks)
    dt : float
        Time step for output grid (biweeks)

    Returns
    -------
    t_eval : array
        Time points
    sol : OdeResult
        Output from solve_ivp
    """

    # Initial recovered (immune)
    R0 = immune_fraction * N

    # Initial susceptible
    S0 = N - I0 - R0

    # Initial state vector
    y0 = [S0, I0, R0]
```

```
    # Time grid (biweeks)
    t_eval = np.arange(0, t_max + dt, dt)

    # Solve the system
    sol = solve_ivp(
        sir_rhs,
        t_span=(0, t_max),
        y0=y0,
        args=(beta, gamma, N),
        t_eval=t_eval,
        vectorized=False
    )

    return t_eval, sol


# ----------------------------
# Example simulation
# ----------------------------

N = 5000          # population
I0 = 10             # initial infections
immune_fraction = 0.0
beta = 2          # transmission rate
gamma = 1.0         # recovery rate per biweek

t, sol = simulate_sir(N, I0, immune_fraction, beta, gamma)

S, I, R = sol.y


# ----------------------------
# Verification: population conserved
# ----------------------------

total_population = S + I + R
print("Population check (min, max):",
      total_population.min(),
      total_population.max())


# ----------------------------
# Plot results
# ----------------------------

plt.figure(figsize=(8,5))
```

```
plt.plot(t, S, label="Susceptible")
plt.plot(t, I, label="Infectious")
plt.plot(t, R, label="Recovered")

plt.xlabel("Time (biweeks)")
plt.ylabel("Population")
plt.title("SIR Epidemic Model")

plt.legend()
plt.grid(True)

plt.show()
```

```
Population check (min, max): 4999.999999999999 5000.0
```



Follow up Tasks:

- Run the model with different values. (e.g.: 0.8, 1.0, 2.0, 3.0,...)
- When does the infection fail to grow?
- If the infectious period is 4 weeks, what should be?
- Change Initial Infections I0 (1 ,10, 100):
  - Does the peak size change?

  - Does the time to peak change?

- Change Population Size N (1000, 10000, 100000):
  - If you scale the population, does the shape of the epidemic curve change?

- Reproduction Number R0:
  - $R_0 = \frac{\beta}{\gamma}$
  - What happens when R < 1?
  - What happens when R > 1?
  - Try: beta = 0.5, 1.0, 1.5 with fixed gamma = 1

- Immunity & Vaccination:
  - Assume 20% of population is immune initially, how does herd immunity / vaccination affect the peak infection?

1.6 4. Map Model Output to Observations (Incidence via ΔH)

Observed data are **biweekly case counts**, not the state `I(t)`.
We model **incidence** using an accumulator:

- Add `H(t)` with **dH/dt = S I / N**
- Predicted biweekly counts **ΔH** over each biweek interval

1.6.1 ChatGPT Prompt (with the same chat session of the above SIR model chat or copy along the model code)

```
Persona:
You are an expert epidemic modeler.

Context:
- Observed data are biweekly case counts (incidence) can be loaded as:

measles_df = pd.read_csv("data/niamey.csv")

dfA = (
    measles_df
    .loc[measles_df["community"] == "A", ["biweek", "measles"]]
    .dropna(subset=["measles"])
    .copy()
)


  - dfA["biweek"] contains the biweekly time index

  - dfA["measles"] contains observed biweekly incidence (number of cases)
```

```
- SIR model outputs states

Task:

Extend the SIR model by:
- Adding H(t) with dH/dt = beta*S*I/N
- Computing predicted biweekly cases as delta_H aligned to biweeks

Then:
- Plot observed measles incidence (dfA["measles"])

- Overlay simulated biweekly incidence (delta_H) on the same figure

- Use t[1:] when plotting delta_H as len(t) is different from len(delta_h) by 1 unit

Constraints:
- Keep code minimal and readable
- Return the full block code with comments to explain in details.
```

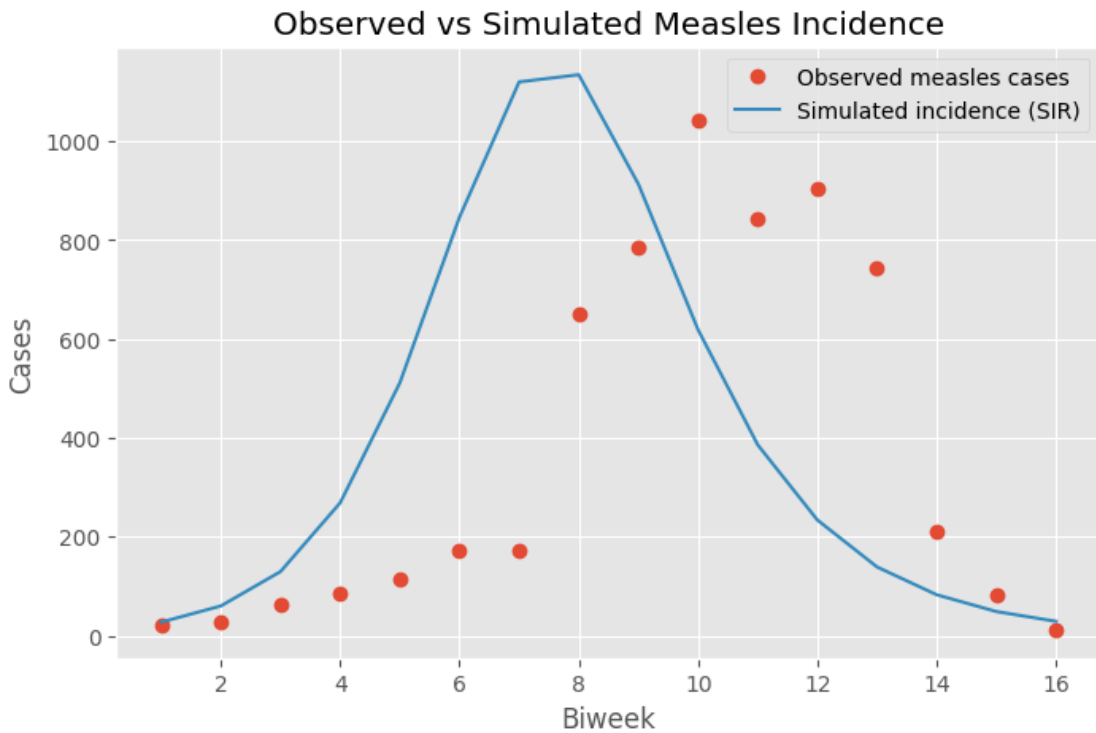
[8]:
```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp


# ------------------------------------------------------------
# Load observed measles incidence
# ------------------------------------------------------------

measles_df = pd.read_csv("data/niamey.csv")

dfA = (
    measles_df
    .loc[measles_df["community"] == "A", ["biweek", "measles"]]
    .dropna(subset=["measles"])
    .copy()
)


# ------------------------------------------------------------
# SIR model with cumulative infections H(t)
# ------------------------------------------------------------

def sir_rhs(t, y, beta, gamma, N):
    """
    Right-hand side of the extended SIR model.
```

```python
    State vector:
        y = [S, I, R, H]

    S : susceptible population
    I : infectious population
    R : recovered population
    H : cumulative infections

    H allows us to compute incidence later.
    """

    S, I, R, H = y

    # force of infection
    infection_rate = beta * S * I / N

    dSdt = -infection_rate
    dIdt = infection_rate - gamma * I
    dRdt = gamma * I

    # cumulative infections
    dHdt = infection_rate

    return [dSdt, dIdt, dRdt, dHdt]


# ------------------------------------------------------------
# Simulation function
# ------------------------------------------------------------

def simulate_sir(N, I0, immune_fraction, beta, gamma, t_max, dt=1):
    """
    Simulate SIR model on a biweekly time grid.

    Parameters
    ----------
    N : population size
    I0 : initial infections
    immune_fraction : fraction immune at start
    beta : transmission rate
    gamma : recovery rate (per biweek)
    t_max : duration of simulation (biweeks)
    dt : output step (biweeks)

    Returns
    -------
    t : time grid
```

```
    sol : solution object
    delta_H : predicted incidence per biweek
    """

    # initial recovered (immune individuals)
    R0 = immune_fraction * N

    # initial susceptible
    S0 = N - I0 - R0

    # cumulative infections initially equal to I0
    H0 = I0

    # state vector
    y0 = [S0, I0, R0, H0]

    # biweekly time grid
    t_eval = np.arange(0, t_max + dt, dt)

    # solve ODE system
    sol = solve_ivp(
        sir_rhs,
        (0, t_max),
        y0,
        args=(beta, gamma, N),
        t_eval=t_eval
    )

    # extract cumulative infections
    H = sol.y[3]

    # compute predicted incidence (biweekly cases)
    delta_H = np.diff(H)

    return t_eval, sol, delta_H


# ------------------------------------------------------------
# Example parameters
# ------------------------------------------------------------

N = 10000
I0 = 10
immune_fraction = 0.1

beta = 2
gamma = 1.0   # infectious period  2 weeks
```

```python
# ------------------------------------------------------------
# Run simulation
# ------------------------------------------------------------

t_max = dfA["biweek"].max()

t, sol, delta_H = simulate_sir(
    N,
    I0,
    immune_fraction,
    beta,
    gamma,
    t_max
)


# ------------------------------------------------------------
# Plot observed vs simulated incidence
# ------------------------------------------------------------

plt.figure(figsize=(8,5))

# observed measles incidence
plt.plot(
    dfA["biweek"],
    dfA["measles"],
    "o",
    label="Observed measles cases"
)

# simulated incidence
# use t[1:] because diff reduces length by 1
plt.plot(
    t[1:],
    delta_H,
    label="Simulated incidence (SIR)"
)

plt.xlabel("Biweek")
plt.ylabel("Cases")
plt.title("Observed vs Simulated Measles Incidence")

plt.legend()
plt.grid(True)
```

```
plt.show()
```



1.7 5. Fit Parameters by Least Squares

We fit parameters by minimizing squared error between observed counts and predicted ΔH.

Remember to edit the interface of the SIR model to match with the generated model in the previous step.

1.7.1 ChatGPT Prompt (copy/paste)

```
## Persona

You are an **expert in numerical optimization for epidemic models**, with experience fitting co

---

## Data

We have **biweekly measles incidence data** for **Community A**:

measles_df = pd.read_csv("data/niamey.csv")

dfA = (
```

```
    measles_df
    .loc[measles_df["community"] == "A", ["biweek", "measles"]]
    .dropna(subset=["measles"])
    .copy()
)

* `dfA["biweek"]` → biweekly time index
* `dfA["measles"]` → observed biweekly incidence (case counts)

---

## Model

We use an SIR model that returns simulated incidence:


def simulate_sir(N, I0, immune_fraction, beta, gamma, t_max, dt=1):
    """
    Simulate SIR model on a biweekly time grid.

    Parameters
    ----------
    N : population size
    I0 : initial infections
    immune_fraction : fraction immune at start
    beta : transmission rate
    gamma : recovery rate (per biweek)
    t_max : duration of simulation (biweeks)
    dt : output step (biweeks)

    Returns
    -------
    t : time grid
    sol : solution object
    delta_H : predicted incidence per biweek
    """
---

## Objective

Fit model parameters to the observed incidence data using **nonlinear least squares**.

### Parameters to estimate:

* `N` (population size)
* `beta` (transmission rate)
* `I0` (initial infected)
* `immune_fraction` (Fraction immune at start)
```

Fixed parameter:

* `gamma = 1.0`

Initialization Strategy

Use an early-growth estimate:

* Estimated basic reproduction number:

  R_0 = 1.443

* Since (R_0 = beta / gamma), initialize beta_0 = 1.443

Choose reasonable initial guesses and bounds:

* `N` within plausible community size in range of (1000, 10000)
* `beta > 0` in the range of (1.0, 10.0)
* `I0  1` in the range of (1,50)
* `immune_fraction` in the range of (0.1,0.4)

Optimization Requirements

* Use `scipy.optimize.least_squares`
* Fit by minimizing SSE = (Observed - Predicted)^2
* Ensure runtime is suitable for a workshop (avoid heavy MCMC or expensive methods)

Verification & Output

After fitting:

1. Overlay observed and fitted incidence curves
2. Print:

   * Best-fit parameter values
   * Sum of squared errors (SSE)

Expected Deliverables

```
* Residual function definition
* Call to `least_squares`
* Overlay plot (observed vs fitted)
* Printed parameter estimates and SSE
* Clean, reproducible workshop-ready code
```

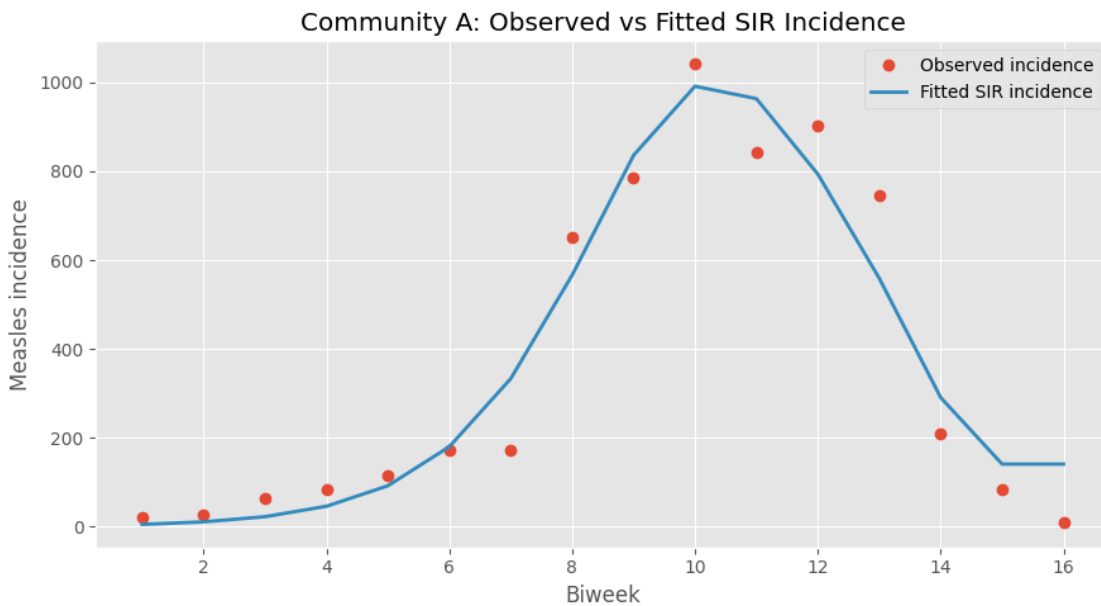
[9]:
```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import least_squares

# -----------------------------
# Load and prepare Community A data
# -----------------------------
measles_df = pd.read_csv("data/niamey.csv")

dfA = (
    measles_df
    .loc[measles_df["community"] == "A", ["biweek", "measles"]]
    .dropna(subset=["measles"])
    .copy()
    .sort_values("biweek")
    .reset_index(drop=True)
)

t_obs = dfA["biweek"].to_numpy(dtype=float)
y_obs = dfA["measles"].to_numpy(dtype=float)

# Fixed parameter
gamma = 1.0

# -----------------------------
# Helper: model prediction at observed biweeks
# -----------------------------
def predict_incidence(params, t_obs, gamma=1.0):
    """
    Return predicted biweekly incidence at the observed time points.

    Parameters
    ----------
    params : iterable
        [N, beta, I0, immune_fraction]
    t_obs : array-like
        Observed biweek time points
    gamma : float
        Recovery rate (fixed)
```

```
    Returns
    -------
    y_pred : np.ndarray
        Predicted incidence aligned to t_obs
    """
    N, beta, I0, immune_fraction = params

    # Guard against invalid values to keep optimizer stable
    if N <= 0 or beta <= 0 or I0 < 1 or not (0 <= immune_fraction <= 1):
        return np.full_like(t_obs, 1e12, dtype=float)

    # Simulation horizon chosen to cover the data
    t_max = int(np.max(t_obs))

    # Assumes simulate_sir is already defined by you
    t_sim, sol, delta_H = simulate_sir(
        N=N,
        I0=I0,
        immune_fraction=immune_fraction,
        beta=beta,
        gamma=gamma,
        t_max=t_max,
        dt=1
    )

    t_sim = np.asarray(t_sim, dtype=float)
    delta_H = np.asarray(delta_H, dtype=float)

    # Interpolate predictions onto the observed biweek grid if needed
    # Align lengths if incidence is one element shorter
    if len(delta_H) == len(t_sim) - 1:
        t_sim = t_sim[:-1]

    y_pred = np.interp(t_obs, t_sim, delta_H)

    # Safety checks
    if np.any(~np.isfinite(y_pred)):
        return np.full_like(t_obs, 1e12, dtype=float)

    return y_pred

# -----------------------------
# Residual function for least squares
# -----------------------------
def residuals(params, t_obs, y_obs, gamma=1.0):
    """
    Residual vector for nonlinear least squares:
```

```
        residuals = observed - predicted
    """
    y_pred = predict_incidence(params, t_obs, gamma=gamma)
    return y_obs - y_pred

# ----------------------------
# Initial guesses and bounds
# ----------------------------
# Early-growth estimate:
#   R0 = 1.443
#   beta0 = R0 * gamma = 1.443 since gamma = 1.0
beta_0 = 1.443

# Reasonable workshop-friendly starting values
N_0 = 5000
I0_0 = max(1.0, y_obs[0] if y_obs[0] > 0 else 5.0)
immune_fraction_0 = 0.1

x0 = np.array([N_0, beta_0, I0_0, immune_fraction_0], dtype=float)

# Bounds:
# - N in plausible community-size range
# - beta > 0
# - I0 >= 1
# - immune_fraction in [0, 1)
lower_bounds = np.array([1000,   0.0, 1.0, 0.0], dtype=float)
upper_bounds = np.array([20000, 10.0, 50.0, 0.9], dtype=float)

# ----------------------------
# Fit with nonlinear least squares
# ----------------------------
fit = least_squares(
    fun=residuals,
    x0=x0,
    bounds=(lower_bounds, upper_bounds),
    args=(t_obs, y_obs, gamma),
    method="trf",      # robust bounded optimizer
    loss="linear"      # ordinary least squares
)

# ----------------------------
# Extract fitted values
# ----------------------------
N_hat, beta_hat, I0_hat, immune_hat = fit.x
y_fit = predict_incidence(fit.x, t_obs, gamma=gamma)
sse = np.sum((y_obs - y_fit) ** 2)
```

```python
# ----------------------------
# Print results
# ----------------------------
print("Best-fit parameter values")
print(f"  N               = {N_hat:.3f}")
print(f"  beta            = {beta_hat:.6f}")
print(f"  I0              = {I0_hat:.3f}")
print(f"  immune_fraction = {immune_hat:.6f}")
print(f"  gamma           = {gamma:.3f}  (fixed)")
print(f"  SSE             = {sse:.3f}")

# ----------------------------
# Overlay plot: observed vs fitted
# ----------------------------
plt.figure(figsize=(9, 5))
plt.plot(t_obs, y_obs, "o", label="Observed incidence")
plt.plot(t_obs, y_fit, "-", linewidth=2, label="Fitted SIR incidence")
plt.xlabel("Biweek")
plt.ylabel("Measles incidence")
plt.title("Community A: Observed vs Fitted SIR Incidence")
plt.legend()
plt.tight_layout()
plt.show()
```

```
Best-fit parameter values
  N               = 9217.499
  beta            = 1.739060
  I0              = 1.058
  immune_fraction = 0.011313
  gamma           = 1.000  (fixed)
  SSE             = 130033.410
```



1.7.2 Epidemiological Interpretation

- What is the estimated basic reproduction number? $R_0 = \frac{\beta}{\gamma}$
  - Is it reasonable for measles?
- What proportion of the population was initially susceptible?
- Does the fitted population size N seem realistic for the community?
- What fraction of the population gets infected during the epidemic? $AttackRate = \frac{H_{final}}{N}$

1.7.3 Things to explore later (optional)

- How sensitive are results to the initial guess? local vs global optima.
- Try different starting values for:
  - N
  - beta
  - I_0
  - immune_fraction
- Fix `N` and refit the model.
  - Does the fit improve or worsen?
- What happens if you fix immune_fraction and only fit and I ?
- Which parameter affects peak height the most?

- The effects of lower bounds and upper bounds
- Repeat the process for community B and C.

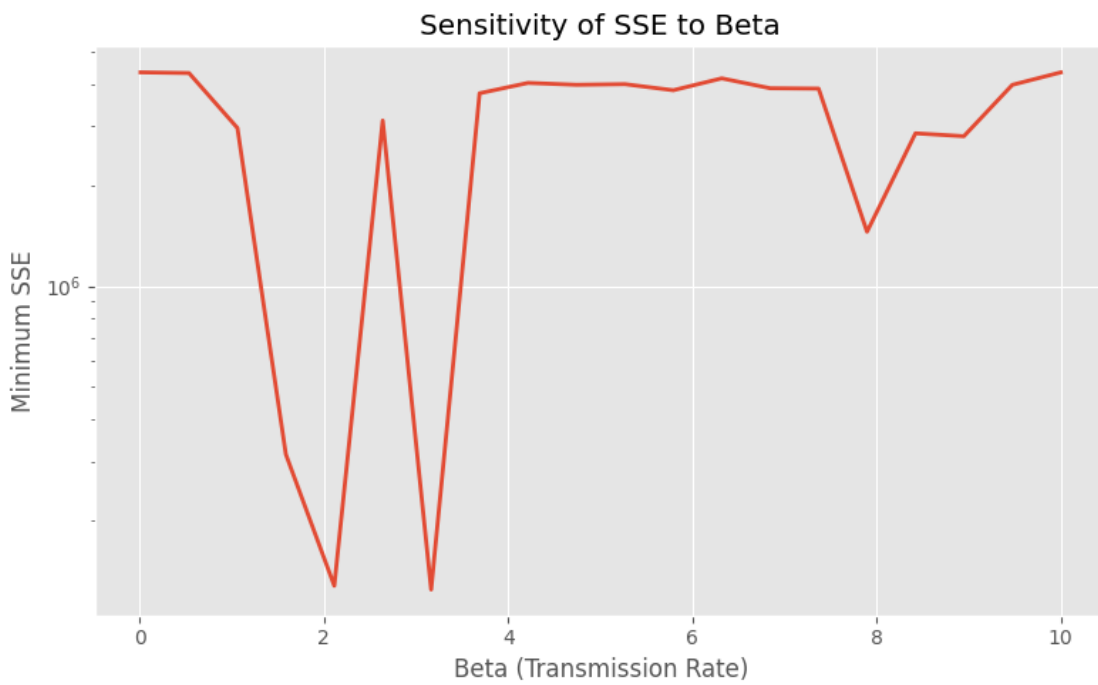
1.8 Sensitivity of fixed beta on min SSE (optional)

Write a full detail prompt based on this is the basic prompt as your homework, code below just for example

`generate code to see sensitivity of beta (from 0 to 10) on min SSE`

[10]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import least_squares

# Range of beta values
beta_grid = np.linspace(0.01, 10, 20)

sse_values = []

for beta_fixed in beta_grid:

    # Residual function where beta is fixed
    def residuals_fixed_beta(params):
        N, I0, immune_fraction = params
        y_pred = predict_incidence([N, beta_fixed, I0, immune_fraction], t_obs,␣
 ↪gamma)
        return y_obs - y_pred

    # Initial guesses
    x0 = [5000, 10, 0.1]

    # Bounds
    lower = [1000, 1, 0.0]
    upper = [20000, 500, 0.9]

    try:
        fit = least_squares(
            residuals_fixed_beta,
            x0=x0,
            bounds=(lower, upper),
            method="trf"
        )

        res = residuals_fixed_beta(fit.x)
        sse = np.sum(res**2)
    except:
        sse = np.nan
```

```
    sse_values.append(sse)

sse_values = np.array(sse_values)

plt.figure(figsize=(8,5))

plt.plot(beta_grid, sse_values, linewidth=2)
plt.xlabel("Beta (Transmission Rate)")
plt.ylabel("Minimum SSE")
#log
plt.yscale("log")
plt.title("Sensitivity of SSE to Beta")
plt.grid(True)

plt.tight_layout()
plt.show()


best_idx = np.nanargmin(sse_values)

print("Best beta from sensitivity scan:", beta_grid[best_idx])
print("Minimum SSE:", sse_values[best_idx])
```




```
Best beta from sensitivity scan: 3.1647368421052633
Minimum SSE: 124389.25925440964
```

1.9 6. Poisson Likelihood Inference (Counts)

For count data: - ($y_t \sim \text{Poisson}(\mu_t)$) - ($\mu_t = \rho \cdot \Delta H_t$)

where is a reporting/ascertainment fraction.

1.9.1 ChatGPT Prompt (copy/paste)

```
# Persona

You are an **expert in numerical optimization for epidemic models**, with experience fitting co

---

# Data

We have **biweekly measles incidence data** for **Community A**:

measles_df = pd.read_csv("data/niamey.csv")

dfA = (
    measles_df
    .loc[measles_df["community"] == "A", ["biweek", "measles"]]
    .dropna(subset=["measles"])
    .copy()
)

t_obs = dfA["biweek"].values
y_obs = dfA["measles"].values


* `t_obs` → biweekly time index
* `y_obs` → observed biweekly incidence (case counts)

---

# Model

We use an SIR model that returns simulated incidence:


def simulate_sir(N, I0, immune_fraction, beta, gamma, t_max, dt=1):
    """
    Simulate SIR model on a biweekly time grid.

    Parameters
    ----------
    N : population size
    I0 : initial infections
```

```
    immune_fraction : fraction immune at start
    beta : transmission rate
    gamma : recovery rate (per biweek)
    t_max : duration of simulation (biweeks)
    dt : output step (biweeks)

    Returns
    -------
    t : time grid
    sol : solution object
    delta_H : predicted incidence per biweek
    """

---

# Objective

Fit model parameters to the observed incidence data using **maximum likelihood under a Poisson

Assume a Poisson observation model:

Y_t ~ Poisson(p *  _t)

where:
- Y_t = observed incidence
-  _t = model-predicted incidence (delta_H)
- p = reporting probability

---
Parameters to Estimate

N : population size
beta : transmission rate
I0 : initial infections
immune_fraction : fraction immune at start

Observation parameter:

p : reporting probability

### Fixed parameter:

* `gamma = 1.0`

---

# Initialization Strategy
```

```
Use early-growth estimate: R_0 = 1.443

Since: R_0 = beta / gamma

Initialize: beta_0 = 1.443

Use reasonable bounds:

* `N` within plausible community size in range of (1000, 10000)
* `beta > 0` in the range of (1.0, 5.0)
* `I0  1` in the range of (1,50)
* `immune_fraction` in the range of (0,0.4)
* p reporting probability in the range of (0.8,1.0)

---

# Optimization Requirements

Use `scipy.optimize.minimize` using Powell method.

Minimize the **negative Poisson log-likelihood**:

-log L(theta) = sum_t (p*lambda_t - y_t * log(p*lambda_t)

(omit constant log(y_t!) term)

- Add small epsilon to avoid `log(0)`.

- The SIR simulator may return incidence values that correspond to intervals between time poin

if len(delta_H) == len(t_sim) - 1:
        t_sim = t_sim[:-1]

- Ensure runtime is suitable for a workshop (no MCMC).

---

# Verification & Output

After fitting:

1. Overlay observed and fitted incidence curves
2. Print:

   * Best-fit parameter values
   * Final negative log-likelihood

---
```

```
# Expected Deliverables

* Negative log-likelihood function definition
* Call to optimizer
* Overlay plot (observed vs fitted incidence)
* Printed parameter estimates
* Clean, reproducible workshop-ready code
* Proper numerical safeguards (`epsilon` for stability)
```

[11]:
```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import minimize

# ---------------------------------------------------------------------
# Data
# ---------------------------------------------------------------------
measles_df = pd.read_csv("data/niamey.csv")

dfA = (
    measles_df
    .loc[measles_df["community"] == "A", ["biweek", "measles"]]
    .dropna(subset=["measles"])
    .copy()
)

t_obs = dfA["biweek"].values.astype(float)
y_obs = dfA["measles"].values.astype(float)

# Fixed recovery rate
gamma = 1.0

# ---------------------------------------------------------------------
# Helper: run simulator and align model incidence to observed times
# ---------------------------------------------------------------------
def get_model_incidence(params, t_obs, gamma=1.0):
    """
    Run SIR simulator and return model-predicted incidence aligned to t_obs.

    Parameters
    ----------
    params : iterable
        (N, beta, I0, immune_fraction, p)
    t_obs : array-like
        Observed biweekly time points
    gamma : float
```

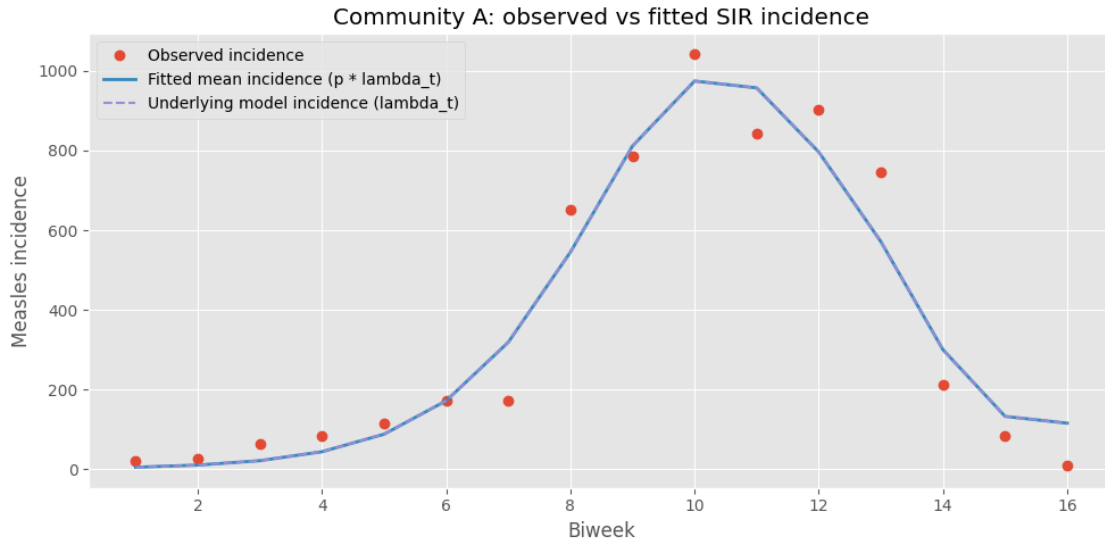
```
        Fixed recovery rate

    Returns
    -------
    mu : ndarray
        Mean Poisson intensities p * lambda_t aligned to t_obs
    lambda_t : ndarray
        Model incidence (before reporting) aligned to t_obs
    """
    N, beta, I0, immune_fraction, p = params

    # Basic feasibility checks
    if N <= 0 or beta <= 0 or I0 < 1 or not (0 <= immune_fraction < 1) or not␣
↪(0 < p <= 1):
        raise ValueError("Parameters outside feasible region.")

    # Simulation duration:
    # include the largest observed biweek and one extra step for safety
    t_max = int(np.max(t_obs)) + 1

    t_sim, sol, delta_H = simulate_sir(
        N=N,
        I0=I0,
        immune_fraction=immune_fraction,
        beta=beta,
        gamma=gamma,
        t_max=t_max,
        dt=1
    )

    t_sim = np.asarray(t_sim, dtype=float)
    delta_H = np.asarray(delta_H, dtype=float)

    # Some simulators return incidence over intervals, so len(delta_H) =␣
↪len(t_sim) - 1
    if len(delta_H) == len(t_sim) - 1:
        t_sim = t_sim[:-1]

    if len(delta_H) != len(t_sim):
        raise ValueError("Simulator output lengths are inconsistent.")

    # Numerical guard: incidence cannot be negative
    delta_H = np.maximum(delta_H, 0.0)

    # Align simulated incidence to observation times
    # If t_obs exactly matches t_sim grid, this is exact; otherwise␣
↪interpolation is used.
```

```python
    lambda_t = np.interp(t_obs, t_sim, delta_H)

    # Apply reporting probability
    mu = p * lambda_t

    return mu, lambda_t

# ---------------------------------------------------------------------
# Negative Poisson log-likelihood
# ---------------------------------------------------------------------
def neg_poisson_loglik(params, t_obs, y_obs, gamma=1.0, epsilon=1e-10):
    """
    Negative Poisson log-likelihood (up to additive constant):

        -log L(theta) = sum_t [mu_t - y_t * log(mu_t)]

    where mu_t = p * lambda_t

    The log(y_t!) term is omitted since it does not depend on parameters.
    """
    N, beta, I0, immune_fraction, p = params

    # Soft bounds / penalties for Powell
    if not (1000 <= N <= 10000):
        return 1e20
    if not (1.0 <= beta <= 5.0):
        return 1e20
    if not (1 <= I0 <= 50):
        return 1e20
    if not (0.0 <= immune_fraction <= 0.4):
        return 1e20
    if not (0.8 <= p <= 1.0):
        return 1e20

    try:
        mu, _ = get_model_incidence(params, t_obs, gamma=gamma)

        # Stability safeguard for log(0)
        mu_safe = np.maximum(mu, epsilon)

        nll = np.sum(mu_safe - y_obs * np.log(mu_safe))

        # Extra safeguard
        if not np.isfinite(nll):
            return 1e20

        return nll
```

```python
    except Exception:
        return 1e20

# ---------------------------------------------------------------------
# Initial values
# ---------------------------------------------------------------------
# Early-growth estimate: R0 = beta / gamma = 1.443, with gamma = 1
beta_0 = 1.443

x0 = np.array([
    5000.0,   # N
    beta_0,   # beta
    5.0,      # I0
    0.10,     # immune_fraction
    0.90      # p
])

bounds = [
    (1000, 10000),  # N
    (1.0, 10.0),     # beta
    (1.0, 50.0),    # I0
    (0.0, 0.9),     # immune_fraction
    (0.01, 1.0)      # p
]

# ---------------------------------------------------------------------
# Optimization
# ---------------------------------------------------------------------
result = minimize(
    fun=neg_poisson_loglik,
    x0=x0,
    args=(t_obs, y_obs, gamma),
    method="Powell",
    bounds=bounds,
    options={
        "maxiter": 300,
        "xtol": 1e-4,
        "ftol": 1e-4,
        "disp": True
    }
)

# ---------------------------------------------------------------------
# Extract fit
# ---------------------------------------------------------------------
N_hat, beta_hat, I0_hat, immune_hat, p_hat = result.x
```

```python
final_nll = result.fun

mu_hat, lambda_hat = get_model_incidence(result.x, t_obs, gamma=gamma)

# ------------------------------------------------------------------
# Print results
# ------------------------------------------------------------------
print("\nBest-fit parameter estimates")
print(f"N                 = {N_hat:.3f}")
print(f"beta              = {beta_hat:.6f}")
print(f"I0                = {I0_hat:.3f}")
print(f"immune_fraction   = {immune_hat:.6f}")
print(f"p                 = {p_hat:.6f}")
print(f"gamma (fixed)     = {gamma:.3f}")
print(f"Final NLL         = {final_nll:.6f}")
print(f"Optimizer success: {result.success}")
print(f"Message: {result.message}")

# ------------------------------------------------------------------
# Plot: observed vs fitted incidence
# ------------------------------------------------------------------
plt.figure(figsize=(10, 5))
plt.plot(t_obs, y_obs, "o", label="Observed incidence")
plt.plot(t_obs, mu_hat, "-", lw=2, label="Fitted mean incidence (p * lambda_t)")
plt.plot(t_obs, lambda_hat, "--", lw=1.5, label="Underlying model incidence␣
↪(lambda_t)")
plt.xlabel("Biweek")
plt.ylabel("Measles incidence")
plt.title("Community A: observed vs fitted SIR incidence")
plt.legend()
plt.tight_layout()
plt.show()
```

```
Optimization terminated successfully.
         Current function value: -31886.036466
         Iterations: 9
         Function evaluations: 1303

Best-fit parameter estimates
N                 = 9291.207
beta              = 1.769446
I0                = 1.018
immune_fraction   = 0.029139
p                 = 0.999934
gamma (fixed)     = 1.000
Final NLL         = -31886.036466
Optimizer success: True
Message: Optimization terminated successfully.
```



1.10 7. Quick Uncertainty: Profile Likelihood for (Optional)

1.10.1 ChatGPT Prompt (copy/paste, continue with the likelihood chat)

```
Persona:
You are an expert in likelihood-based inference.

Context:
- We already have a Poisson likelihood fit
- We want uncertainty for beta

Task:
Create a profile likelihood for beta:
1) Fix beta over a grid range (1.0, 5.0)
2) Optimize N, I0, immune_fraction, and p for each beta

* `N` within plausible community size range (1000, 10000)
* `I0  1` (1,50)
* `immune_fraction` in the range of (0,0.4)
* p reporting probability in the range of (0.8,1.0)

3) Plot profile NLL(beta)

Constraints:
- Keep runtime reasonable for a workshop

Output:
- Profile plot
- Explain how to interpret the uncertainty
```

- Printed parameter estimates
- Single code block for Google Colab

[12]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import minimize

# ------------------------------------------------------------
# Load data
# ------------------------------------------------------------
measles_df = pd.read_csv("data/niamey.csv")

dfA = (
    measles_df
    .loc[measles_df["community"] == "A", ["biweek", "measles"]]
    .dropna(subset=["measles"])
    .copy()
)

t_obs = dfA["biweek"].values.astype(float)
y_obs = dfA["measles"].values.astype(float)

gamma = 1.0
epsilon = 1e-10


# ------------------------------------------------------------
# Helper: simulate model incidence aligned to observation grid
# ------------------------------------------------------------
def get_model_incidence(params):
    N, beta, I0, immune_fraction, p = params

    t_max = int(np.max(t_obs)) + 1

    t_sim, sol, delta_H = simulate_sir(
        N=N,
        I0=I0,
        immune_fraction=immune_fraction,
        beta=beta,
        gamma=gamma,
        t_max=t_max,
        dt=1
    )

    t_sim = np.asarray(t_sim)
    delta_H = np.asarray(delta_H)
```

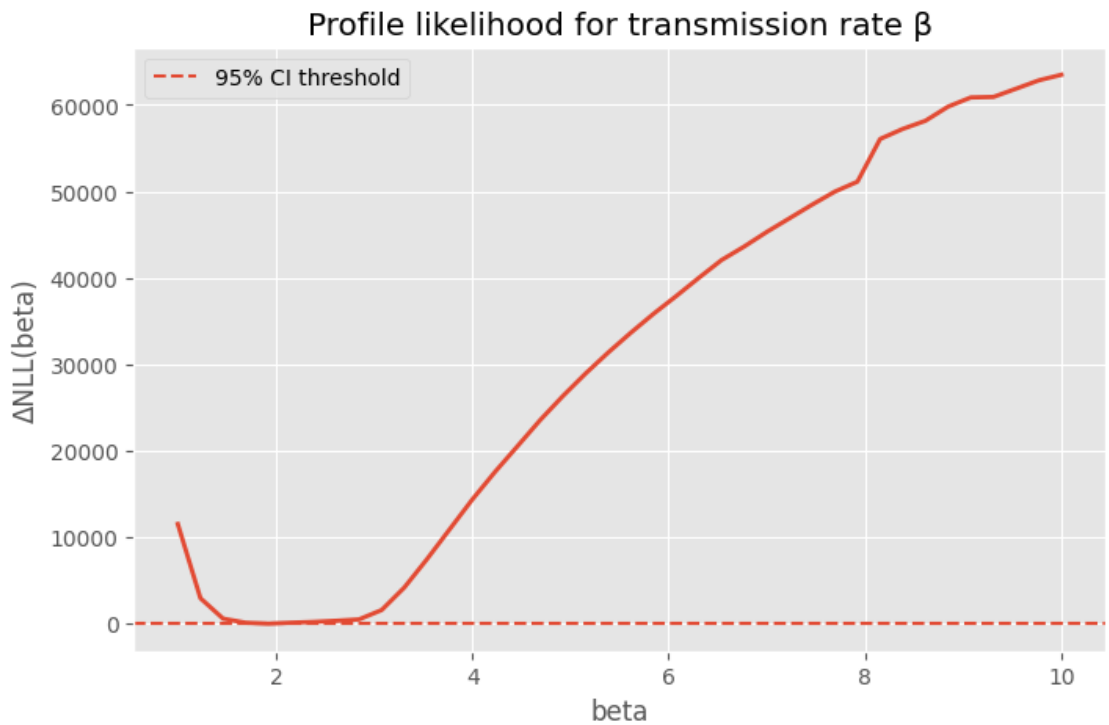
```python
    # handle interval incidence
    if len(delta_H) == len(t_sim) - 1:
        t_sim = t_sim[:-1]

    delta_H = np.maximum(delta_H, 0)

    lambda_t = np.interp(t_obs, t_sim, delta_H)

    mu = p * lambda_t

    return mu, lambda_t


# ------------------------------------------------------------
# Negative Poisson log-likelihood
# ------------------------------------------------------------
def neg_poisson_loglik(params):

    N, beta, I0, immune_fraction, p = params

    # enforce bounds via penalty (Powell friendly)
    if not (1000 <= N <= 10000):
        return 1e20
    if not (1 <= I0 <= 50):
        return 1e20
    if not (0 <= immune_fraction <= 0.4):
        return 1e20
    if not (0.8 <= p <= 1.0):
        return 1e20
    if not (beta > 0):
        return 1e20

    try:
        mu, _ = get_model_incidence(params)

        mu_safe = np.maximum(mu, epsilon)

        nll = np.sum(mu_safe - y_obs * np.log(mu_safe))

        if not np.isfinite(nll):
            return 1e20

        return nll

    except:
        return 1e20
```

```python
# ------------------------------------------------------------
# Fit full model once (for reference MLE)
# ------------------------------------------------------------
x0 = [5000, 1.443, 5, 0.1, 0.9]

result = minimize(
    neg_poisson_loglik,
    x0,
    method="Powell",
    options={"maxiter": 300}
)

N_hat, beta_hat, I0_hat, immune_hat, p_hat = result.x
mle_nll = result.fun

print("===== Full MLE =====")
print(f"N = {N_hat:.2f}")
print(f"beta = {beta_hat:.4f}")
print(f"I0 = {I0_hat:.2f}")
print(f"immune_fraction = {immune_hat:.3f}")
print(f"p = {p_hat:.3f}")
print(f"NLL = {mle_nll:.3f}")


# ------------------------------------------------------------
# Profile likelihood for beta
# ------------------------------------------------------------
beta_grid = np.linspace(1.0, 10.0, 40)

profile_nll = []
profile_params = []

for beta_fixed in beta_grid:

    def obj_profile(theta):
        N, I0, immune_fraction, p = theta
        return neg_poisson_loglik([N, beta_fixed, I0, immune_fraction, p])

    x0_profile = [N_hat, I0_hat, immune_hat, p_hat]

    res = minimize(
        obj_profile,
        x0_profile,
        method="Powell",
        options={"maxiter": 200}
```

```
    )

    profile_nll.append(res.fun)
    profile_params.append(res.x)

    print(f"beta={beta_fixed:.2f}  NLL={res.fun:.3f}")


profile_nll = np.array(profile_nll)


# ------------------------------------------------------------
# Convert to ΔNLL for easier interpretation
# ------------------------------------------------------------
delta_nll = profile_nll - np.min(profile_nll)


# ------------------------------------------------------------
# Plot profile likelihood
# ------------------------------------------------------------
plt.figure(figsize=(8,5))
plt.plot(beta_grid, delta_nll, lw=2)
plt.axhline(1.92, linestyle="--", label="95% CI threshold")
plt.xlabel("beta")
plt.ylabel("ΔNLL(beta)")
plt.title("Profile likelihood for transmission rate  ")
plt.legend()
plt.show()


# ------------------------------------------------------------
# Print parameter estimates along the profile
# ------------------------------------------------------------
print("\n===== Profile Parameter Estimates =====")
for b, params, nll in zip(beta_grid, profile_params, profile_nll):
    N, I0, immune, p = params
    print(f"beta={b:.2f} | N={N:.1f}  I0={I0:.2f}  immune={immune:.3f}  p={p:.
 ↪3f}  NLL={nll:.3f}")
```

```
===== Full MLE =====
N = 9595.43
beta = 1.7589
I0 = 1.08
immune_fraction = 0.018
p = 0.959
NLL = -31885.440
beta=1.00  NLL=-20361.754
beta=1.23  NLL=-28944.179
```

```
beta=1.46  NLL=-31306.550
beta=1.69  NLL=-31776.918
beta=1.92  NLL=-31890.890
beta=2.15  NLL=-31771.804
beta=2.38  NLL=-31682.214
beta=2.62  NLL=-31551.379
beta=2.85  NLL=-31390.821
beta=3.08  NLL=-30305.670
beta=3.31  NLL=-27699.862
beta=3.54  NLL=-24422.133
beta=3.77  NLL=-20971.172
beta=4.00  NLL=-17524.806
beta=4.23  NLL=-14338.194
beta=4.46  NLL=-11341.557
beta=4.69  NLL=-8328.719
beta=4.92  NLL=-5554.582
beta=5.15  NLL=-2955.369
beta=5.38  NLL=-524.194
beta=5.62  NLL=1769.428
beta=5.85  NLL=3951.573
beta=6.08  NLL=5998.712
beta=6.31  NLL=8110.575
beta=6.54  NLL=10161.744
beta=6.77  NLL=11725.421
beta=7.00  NLL=13420.214
beta=7.23  NLL=14993.360
beta=7.46  NLL=16571.915
beta=7.69  NLL=18076.865
beta=7.92  NLL=19231.237
beta=8.15  NLL=24165.699
beta=8.38  NLL=25329.742
beta=8.62  NLL=26269.569
beta=8.85  NLL=27906.054
beta=9.08  NLL=28965.389
beta=9.31  NLL=29024.851
beta=9.54  NLL=29978.135
beta=9.77  NLL=30942.952
beta=10.00  NLL=31591.770
```




```
===== Profile Parameter Estimates =====
beta=1.00 | N=10000.0  I0=50.00  immune=0.000  p=1.000  NLL=-20361.754
beta=1.23 | N=10000.0  I0=50.00  immune=0.000  p=1.000  NLL=-28944.179
beta=1.46 | N=9999.9  I0=10.68  immune=0.000  p=1.000  NLL=-31306.550
beta=1.69 | N=9550.0  I0=1.86  immune=0.011  p=0.971  NLL=-31776.918
beta=1.92 | N=9992.8  I0=1.00  immune=0.101  p=1.000  NLL=-31890.890
beta=2.15 | N=10000.0  I0=1.00  immune=0.197  p=1.000  NLL=-31771.804
beta=2.38 | N=10000.0  I0=1.00  immune=0.276  p=1.000  NLL=-31682.214
beta=2.62 | N=9999.9  I0=1.00  immune=0.341  p=1.000  NLL=-31551.379
beta=2.85 | N=10000.0  I0=1.00  immune=0.396  p=1.000  NLL=-31390.821
beta=3.08 | N=10000.0  I0=1.00  immune=0.400  p=1.000  NLL=-30305.670
beta=3.31 | N=10000.0  I0=1.00  immune=0.400  p=1.000  NLL=-27699.862
beta=3.54 | N=10000.0  I0=1.00  immune=0.400  p=1.000  NLL=-24422.133
beta=3.77 | N=10000.0  I0=1.00  immune=0.400  p=1.000  NLL=-20971.172
beta=4.00 | N=10000.0  I0=1.00  immune=0.400  p=1.000  NLL=-17524.806
beta=4.23 | N=10000.0  I0=1.00  immune=0.400  p=1.000  NLL=-14338.194
beta=4.46 | N=10000.0  I0=1.00  immune=0.400  p=1.000  NLL=-11341.557
beta=4.69 | N=10000.0  I0=1.00  immune=0.400  p=1.000  NLL=-8328.719
beta=4.92 | N=10000.0  I0=1.00  immune=0.400  p=1.000  NLL=-5554.582
beta=5.15 | N=10000.0  I0=1.00  immune=0.400  p=1.000  NLL=-2955.369
beta=5.38 | N=10000.0  I0=1.00  immune=0.400  p=1.000  NLL=-524.194
beta=5.62 | N=10000.0  I0=1.00  immune=0.400  p=1.000  NLL=1769.428
beta=5.85 | N=10000.0  I0=1.00  immune=0.400  p=1.000  NLL=3951.573
```

```
beta=6.08 | N=10000.0  I0=1.00  immune=0.400  p=1.000  NLL=5998.712
beta=6.31 | N=9267.8  I0=1.00  immune=0.400  p=1.000  NLL=8110.575
beta=6.54 | N=8626.2  I0=1.00  immune=0.400  p=1.000  NLL=10161.744
beta=6.77 | N=9874.4  I0=1.08  immune=0.400  p=1.000  NLL=11725.421
beta=7.00 | N=9944.8  I0=1.08  immune=0.400  p=1.000  NLL=13420.214
beta=7.23 | N=10000.0  I0=1.00  immune=0.400  p=1.000  NLL=14993.360
beta=7.46 | N=9795.1  I0=1.09  immune=0.400  p=1.000  NLL=16571.915
beta=7.69 | N=9019.4  I0=1.00  immune=0.400  p=1.000  NLL=18076.865
beta=7.92 | N=10000.0  I0=1.00  immune=0.400  p=1.000  NLL=19231.237
beta=8.15 | N=9999.3  I0=16.11  immune=0.400  p=1.000  NLL=24165.699
beta=8.38 | N=9543.6  I0=24.46  immune=0.400  p=1.000  NLL=25329.742
beta=8.62 | N=9575.3  I0=24.53  immune=0.400  p=1.000  NLL=26269.569
beta=8.85 | N=7923.5  I0=22.38  immune=0.395  p=1.000  NLL=27906.054
beta=9.08 | N=7601.3  I0=22.06  immune=0.395  p=1.000  NLL=28965.389
beta=9.31 | N=10000.0  I0=20.23  immune=0.400  p=1.000  NLL=29024.851
beta=9.54 | N=10000.0  I0=21.11  immune=0.400  p=1.000  NLL=29978.135
beta=9.77 | N=10000.0  I0=9.54  immune=0.395  p=1.000  NLL=30942.952
beta=10.00 | N=10000.0  I0=11.94  immune=0.400  p=1.000  NLL=31591.770
```

1.11 8. Generate a Markdown Summary (Optional)

Participants are encouraged to modify these sample prompts to align with their preferences.

1.11.1 Prompt 1 - Full report

```
Persona

You are an expert epidemiologist, infectious disease modeler, and scientific report writer.

Your task is to help write a clear scientific report describing an epidemic modeling analysis p

The audience is graduate students learning infectious disease modeling.

Write in clear academic language but keep it concise and readable.

Do not invent numerical results. If values are not provided, leave placeholders.

---

Context

We analyzed measles outbreak data from Niamey, Niger.

Dataset:

• Biweekly measles case counts
• Communities: A, B, C
• Time unit: biweeks
```

The analysis included:

- Data visualization
- Early exponential growth estimation
- Estimation of the basic reproduction number (R)
- Implementation of a deterministic SIR model
- Mapping model states to observed incidence
- Parameter estimation using nonlinear least squares
- Parameter estimation using Poisson likelihood
- Sensitivity analysis
- Profile likelihood analysis

Assumptions used in the model:

- Infectious period 2 weeks
- Therefore = 1 per biweek
- Basic reproduction number:

R = /

Observed data represent biweekly incidence.

Model incidence is computed as:

ΔH(t) = cumulative infections difference per biweek

Observation model:

Y ~ Poisson(p ·)

Where

  = model-predicted incidence
p = reporting probability

Results from the modeling exercise

(Participants should paste their results here)

Estimated parameters:

  = ___
R = ___
N = ___
I = ___

```
immune_fraction = ___
reporting probability p = ___

Early outbreak growth rate:

r = ___

Attack rate:

AttackRate = ___

Key observations from the epidemic curves:

• ___
• ___
• ___

---

Task

Write a complete modeling report with the following sections:

1. Introduction
2. Data Description
3. Early Outbreak Analysis
4. SIR Model
5. Parameter Estimation
6. Results
7. Sensitivity Analysis
8. Uncertainty Analysis
9. Comparison Between Communities
10. Limitations
11. Conclusion

---

Output requirements

• Write in Markdown
• Use LaTeX for equations
• Use section headings
• Use bullet points where appropriate
• Do not generate code
• Focus on interpretation and explanation
```

1.11.2 Prompt 2 — Generate the Introduction Section

```
Persona

You are an infectious disease modeler and scientific writer.

Write a short introduction for a modeling report analyzing a measles outbreak using an SIR mode

Context

The analysis uses biweekly measles case data from three communities in Niamey, Niger.

The study aims to:

• estimate early epidemic growth
• estimate the basic reproduction number (R )
• build and fit a deterministic SIR model
• estimate transmission parameters
• analyze model sensitivity and uncertainty

Task

Write a clear academic introduction that explains:

• measles epidemiology
• why SIR models are useful
• the purpose of the analysis

Length

150-250 words.
```

1.11.3 Prompt 3 — Interpret Parameter Estimates

```
Persona

You are an expert epidemiologist.

Explain the epidemiological meaning of the following SIR model parameter estimates.

Parameters:

  = ___
R  = ___
N = ___
I  = ___
immune_fraction = ___
reporting probability p = ___
```

Context

The model describes a measles outbreak using biweekly incidence data.

Task

Explain:

- what each parameter represents
- whether the value is epidemiologically reasonable
- what it implies about transmission dynamics

Write in clear scientific language suitable for a modeling report.

1.11.4 Prompt 4 - Interpret Sensitivity Analysis

Persona

You are an expert infectious disease modeler.

Explain the interpretation of the following sensitivity analyses performed on an SIR model.

Analysis 1

R estimated using different early outbreak windows.

Analysis 2

Sum of squared errors plotted as a function of .

Task

Explain:

- why these analyses are performed
- what stability of R across windows means
- what a flat vs sharp SSE curve means
- what this tells us about parameter identifiability

Write the explanation for a scientific report.

1.11.5 Prompt 5 - Interpret Profile Likelihood

Persona

You are an expert in statistical inference for infectious disease models.

```
Explain profile likelihood analysis for the transmission rate  in an SIR model.

Context

The analysis fixes  at different values and re-optimizes other parameters to compute the likel

Task

Explain:

• why profile likelihood is used
• how confidence intervals are obtained
• how to interpret flat vs sharp likelihood curves

Write the explanation in clear academic language.
```

1.11.6 Prompt 6 - Write the Conclusion

```
Persona

You are an epidemiological modeling expert.

Write a short conclusion for a modeling report analyzing measles outbreak dynamics using an SI

The report included:

• early outbreak analysis
• estimation of R
• deterministic SIR modeling
• parameter estimation
• sensitivity analysis
• uncertainty analysis

Task

Summarize:

• key findings
• usefulness of mechanistic epidemic models
• role of AI-assisted workflows in accelerating modeling analysis

Length

120-200 words.
```

[12]: